

CKKS 参数选择教学文档：从 N, n, Q, P, Δ, L 到安全版 no- B_k 实验

整理稿

2026 年 4 月 29 日

摘要

本文档用于解释 CKKS 与 bootstrapping 实验中最常见的参数：环维度 N 、拆分后的 leaf 维度 n 、拆分因子 k 、密文模数链 Q 、key-switching 辅助模数 P 、最大公开模数 QP 、scale Δ 、level 数 L 、slot 数以及 bit security 之间的关系。本文档的目标不是给出最终密码学安全证明，而是建立一套工程上可用的参数选择直觉，并给出当前 no- B_k coefficient-split bootstrapping 实验应如何选安全参数。

目录

1	最重要的结论	3
2	CKKS 的几个核心参数	3
2.1	N : ring degree	3
2.2	n 与 k : 拆分后的 leaf ring	4
2.3	Q : ciphertext modulus chain	4
2.4	P : key switching auxiliary modulus	5
2.5	Δ : CKKS scale	5
2.6	L : level / multiplicative depth	6
3	安全性依赖哪些数字?	6
3.1	为什么 q 越大安全性越低?	7
4	128-bit 安全的参考表	7
4.1	最重要的近似规律	7
5	把安全表套到你的 no-B_k 实验	8
5.1	当前 correctness prototype: $\log N = 16, k = 4, \log n = 14$	8
5.2	安全版 $k = 2$: $\log N = 17, k = 2, \log n = 16$	8
5.3	安全版 $k = 4$: $\log N = 18, k = 4, \log n = 16$	9
6	推荐参数表	10

目录	2
7 为什么 $\log N = 17, k = 2$ 不应该和 $\log N = 16$ 直接比 latency?	10
8 当前实验数据摘要	11
8.1 $\log N = 17, k = 2, \log n = 16$	11
8.2 $\log N = 16, k = 2, \log n = 15$	12
9 参数选择流程：从应用需求到 N, Q, n	12
9.1 Step 1: 确定应用需求	12
9.2 Step 2: 确定 scale Δ	12
9.3 Step 3: 估计 Q	13
9.4 Step 4: 加上 P 得到 QP	13
9.5 Step 5: 根据 $q_{\max}$ 选择 leaf n	13
9.6 Step 6: 根据 k 得到 top N	14
9.7 Step 7: 跑 estimator	14
10 no-B_k 分解中的安全性表达	14
11 参数和安全性的常见误区	15
11.1 误区 1: top N 安全, 所以 leaf n 也安全	15
11.2 误区 2: 只看输出 ciphertext level 的 Q	15
11.3 误区 3: N 越大只是消息更多	15
11.4 误区 4: speedup 应该最多是 k	15
11.5 误区 5: normalization factor 永远是 k	15
12 推荐的实验命令	15
12.1 安全版 $k = 2$	15
12.2 安全版 $k = 4$	16
13 Go 代码：打印参数和安全相关摘要	16
14 最终建议	16
14.1 Correctness / mechanism prototype	17
14.2 Security-aware prototype	17
14.3 Full secure high-throughput target	17
15 一句话总结	18

1 最重要的结论

先给出最实用的结论。

问题	结论
N 是什么?	CKKS 的 ring degree, 通常 $N = 2^{\log N}$ 。
slot 数是多少?	full complex CKKS 通常有 $N/2$ 个 complex slots。
N 越大是不是消息更多?	是, 但更重要的是安全预算更大, 能承受更大的 Q 或 QP 。
Q 是什么?	ciphertext modulus chain 的乘积, 支持 depth、precision 和 bootstrapping。
P 是什么?	key switching / relinearization / rotation keys 的 auxiliary modulus。
安全估计看 Q 还是 QP ?	通常保守看攻击者可见的最大公开模数, bootstrapping keys 常看 QP 。
拆到 leaf 后看谁的安全?	看 leaf ring R_n 上的 $(n, q_{\max})$, 不能自动继承 top N 。
$n = N/k$ 安全预算如何变?	粗略地, 固定 128-bit 时允许的 $\log q$ 约随 n 线性增长。
当前最推荐安全实验	$\log N_{\text{top}} = 17, k = 2, \log n_{\text{leaf}} = 16, \log(QP) \approx 1520$ 。
更大 $k = 4$ 安全实验	$\log N_{\text{top}} = 18, k = 4, \log n_{\text{leaf}} = 16$, 较重。
当前 $\log N = 16, k = 4, n = 2^{14}$	correctness prototype, 不应作为 128-bit 安全实例。

2 CKKS 的几个核心参数

2.1 N : ring degree

CKKS 通常工作在 2-power cyclotomic / negacyclic ring:

$$R_Q = \mathbb{Z}_Q[X]/(X^N + 1), \quad N = 2^{\log N}.$$

在 full complex CKKS 中, 一个 ciphertext 最多可以编码:

$$\frac{N}{2}$$

个 complex slots。

$\log N$	N	complex slots $N/2$	常见用途
13	8192	4096	debug, toy correctness
14	16384	8192	prototype, 小 leaf, 低/中等模数
15	32768	16384	中等 CKKS, 轻量 bootstrapping 或实验 leaf
16	65536	32768	常见 CKKS bootstrapping 安全维度
17	131072	65536	大 batch; 安全 $k = 2$ no- B_k top
18	262144	131072	更大 batch; 安全 $k = 4$ no- B_k top

核心直觉。 N 不是单纯的容量参数，它同时是：

容量参数 + 安全参数 + 性能成本参数.

N 越大：

- slots 越多；
- 可以承受更大的 ciphertext modulus；
- 可以支持更深的计算或 bootstrapping；
- 但 ciphertext、key、NTT、rotation、key-switching 和内存成本都变大。

2.2 n 与 k ：拆分后的 leaf ring

你的 no- B_k coefficient-split 路线是：

$$N \longrightarrow n = \frac{N}{k}.$$

如果 $k = 2^a$ ，则：

$$\log n = \log N - a.$$

例如：

$$\log N = 17, \quad k = 2 \quad \implies \quad \log n = 16.$$

$$\log N = 18, \quad k = 4 \quad \implies \quad \log n = 16.$$

这里最关键的是：

如果 leaf ciphertext / leaf keys 真正在 R_n 上存在，则安全性要按 n 估计。

不能说 top ring N 安全，所以 leaf ring n 也安全。

2.3 Q ：ciphertext modulus chain

CKKS 中：

$$Q = \prod_{i=0}^L q_i.$$

通常每个 q_i 是一个 RNS prime，例如 40-bit、50-bit 或 60-bit prime。

如果有 20 个 50-bit primes：

$$\log_2 Q \approx 1000.$$

Q 的作用：

- 支持更多 level;
- 支持更多乘法和 rescale;
- 支持 bootstrapping 中的 C2S、EvalMod、S2C;
- 增大 Q 会降低 RLWE 安全性。

因此:

$$Q \uparrow \Rightarrow \text{支持更深计算, 但安全性下降, 需要更大 } N.$$

2.4 P : key switching auxiliary modulus

在 RNS CKKS 中, rotation key、relinearization key、bootstrapping key、ring switching key 等通常涉及一个 auxiliary modulus:

$$P = \prod_j p_j.$$

例如常见设置:

$$\log_2 P \approx 4 \times 61 \approx 244,$$

或:

$$\log_2 P \approx 5 \times 61 \approx 305.$$

安全估计时, 经常不能只看 Q , 而要看 evaluation keys 中出现的最大公开模数:

$$q_{\max} \approx QP.$$

这也是你 no- B_k 分解安全分析中最重要的一点:

$$\text{leaf bootstrapping keys 如果在 } R_{n, QP} \text{ 上生成, 则 leaf 安全要看 } (n, QP).$$

2.5 Δ : CKKS scale

CKKS 把 real/complex message 乘上 scale Δ 编码成近似整数:

$$m \mapsto \lfloor \Delta m \rfloor.$$

常见:

$$\Delta = 2^{30}, \quad 2^{35}, \quad 2^{40}.$$

scale 越大:

- 初始精度越高;
- 每次 multiplication 后 scale 变为 Δ^2 ;
- 需要 rescale;
- 每次 rescale 大约消耗一个与 $\log \Delta$ 类似大小的 prime。

所以:

$$\Delta \uparrow \Rightarrow Q \uparrow \Rightarrow N \uparrow.$$

2.6 L : level / multiplicative depth

如果一个 CKKS computation 需要 L 次 rescale, 则 modulus chain 至少需要对应数量的 primes。

粗略地, 如果每层 prime 约 $\log \Delta$ bits, 那么:

$$\log_2 Q \approx \log q_0 + L \log \Delta + \text{special/residual primes}.$$

Bootstrapping 本身也需要很多 level, 例如 C2S、EvalMod、S2C 都会消耗 levels。因此 CKKS bootstrapping 常见的 $\log(QP)$ 会到 1500 bits 左右。

3 安全性依赖哪些数字?

CKKS/RLWE 安全性主要看:

$$N, \quad q_{\max}, \quad \chi_s, \quad \chi_e, \quad \sigma, \quad \text{public samples / keys}.$$

其中最关键的工程量是:

$$N \text{ 和 } \log_2 q_{\max}.$$

在有 evaluation keys 的 bootstrapping 系统中, 保守地:

$$q_{\max} = \max\{Q_{\text{ciphertext}}, Q_{\text{public/eval keys}}, QP_{\text{switching keys}}, QP_{\text{bootstrapping keys}}, QP_{\text{split/merge keys}}\}.$$

通常可以先取:

$$q_{\max} = QP.$$

3.1 为什么 q 越大安全性越低？

RLWE sample 可抽象为：

$$(a, b) = (a, as + e) \pmod{q}.$$

噪声 e 小，模数 q 大，意味着相对噪声比例变小：

$$\frac{\|e\|}{q} \downarrow.$$

攻击者更容易区分或恢复 secret。因此：

$$q \uparrow \Rightarrow \text{安全性下降.}$$

而 N 越大，格维度越高，攻击越困难：

$$N \uparrow \Rightarrow \text{安全性上升.}$$

4 128-bit 安全的参考表

FHE Security Guidelines 给出了一张非常实用的表：在 Gaussian error standard deviation $\sigma = 3.19$ 、uniform ternary secret 或 Gaussian secret 下，不同维度 N 对应 Category 128/192/256 的最大 $\log_2(q)$ 。这里摘取最常用的 128-bit / ternary 行：

Ring degree N	128-bit 下最大 $\log_2(q)$, ternary secret
$2^{10} = 1024$	26
$2^{11} = 2048$	53
$2^{12} = 4096$	106
$2^{13} = 8192$	214
$2^{14} = 16384$	430
$2^{15} = 32768$	868
$2^{16} = 65536$	1747
$2^{17} = 131072$	3523

注意。 这张表不是你的最终安全证明。最终仍要跑 lattice estimator。但它是非常好的工程直觉表。

4.1 最重要的近似规律

从表中可以看到，固定 128-bit 安全时：

$$\log_2 q_{\max}(N) \text{ 大致随 } N \text{ 线性增长.}$$

因此如果：

$$n = \frac{N}{k},$$

那么粗略地:

$$\log_2 q_{\max}(n) \approx \frac{1}{k} \log_2 q_{\max}(N).$$

这就是为什么:

拆到子环后, 如果 leaf 仍用 top 的大 QP , leaf 安全性通常会下降。

5 把安全表套到你的 $no-B_k$ 实验

5.1 当前 correctness prototype: $\log N = 16, k = 4, \log n = 14$

参数:

$$\log N_{\text{top}} = 16, \quad k = 4, \quad \log n_{\text{leaf}} = 14.$$

如果 leaf 使用:

$$\log_2(QP) \approx 1520,$$

但 $n = 2^{14}$ 的 128-bit 参考预算只有:

$$\log_2 q_{\max} \approx 430.$$

因此:

$$1520 \gg 430.$$

所以这组应该定位为:

correctness / performance prototype, 而不是 128-bit secure instantiation.

5.2 安全版 $k = 2$: $\log N = 17, k = 2, \log n = 16$

参数:

$$\log N_{\text{top}} = 17, \quad k = 2, \quad \log n_{\text{leaf}} = 16.$$

如果:

$$\log_2(QP) \approx 1520,$$

而 $n = 2^{16}$ 的 128-bit 预算约为:

1747.

因此:

$$1520 < 1747.$$

这就是目前最推荐的安全感更强的实验设置:

$$\boxed{\log N_{\text{top}} = 17, \quad k = 2, \quad \log n_{\text{leaf}} = 16, \quad \log(QP) \approx 1520.}$$

你的实验中这组的 leaf boot params 是:

$$\log n = 16, \quad \log \text{Slots} = 15, \quad \log QP = 1520.0.$$

同时 top 和 leaf Q/P moduli 完全一致, ring-basis compatibility check 通过。实验还显示 RAW normalization 对齐, 即 $\gamma_{\text{impl}} \approx 1$ 。

5.3 安全版 $k = 4$: $\log N = 18, k = 4, \log n = 16$

如果希望保留 $k = 4$ 的更高并行度, 同时 leaf 仍然使用安全常见的 $\log n = 16$:

$$k = 4, \quad \log n = 16 \quad \Rightarrow \quad \log N_{\text{top}} = 18.$$

这是:

$$\boxed{\log N_{\text{top}} = 18, \quad k = 4, \quad \log n_{\text{leaf}} = 16.}$$

优点:

- leaf 可以承受 $\log(QP) \approx 1500$ 级别的 bootstrapping;
- $k = 4$ 并行潜力更大;
- slots 更多, 适合大 batch / high-throughput 场景。

缺点:

- direct baseline 会非常慢;
- memory、evaluation keys、rotation keys 都会大幅增加;
- 实验可能很重。

6 推荐参数表

名称	top $\log N$	k	leaf $\log n$	$\log(QP)$	定位
P0 prototype	16	4	14	1520	correctness only, 不安全
P1 small demo	16	2	15	1520	correctness/timing, 不安全或低安全
S1 recommended	17	2	16	1520–1555	推荐安全版 $k = 2$
S2 heavy	18	4	16	1520–1555	推荐安全版 $k = 4$, 很重
S3 high capacity	18	4	16	1730–1745	接近 2^{16} leaf 128-bit 上界, 需 estimator

最推荐先跑的参数。

$$\log N_{\text{top}} = 17, \quad k = 2, \quad \log n_{\text{leaf}} = 16, \quad \log_2(QP) \approx 1520.$$

这是最均衡的选择: leaf 维度足够大, correctness 已经跑通, speedup 明显, 实验成本还不至于像 $\log N = 18$ 那么重。

7 为什么 $\log N = 17, k = 2$ 不应该和 $\log N = 16$ 直接比 latency?

$\log N = 17$ 的 slot 数是:

$$2^{16} = 65536.$$

$\log N = 16$ 的 slot 数是:

$$2^{15} = 32768.$$

所以:

$$\log N = 17$$

处理的是两倍容量。直接比较:

$$T(\log N = 17, k = 2) \quad \text{vs} \quad T(\log N = 16)$$

是不公平的。更合理的是比较:

$$T_{\text{factored}}(17, k = 2) \quad \text{vs} \quad T_{\text{direct}}(17),$$

以及 throughput 角度比较:

$$T_{\text{factored}}(17, k = 2) \quad \text{vs} \quad 2T_{\text{direct}}(16).$$

8 当前实验数据摘要

8.1 $\log N = 17, k = 2, \log n = 16$

指标	数值
top $\log N$	17
leaf $\log n$	16
k	2
$\log(QP)$	1520.0
Ring basis compatibility	PASS
Best C2S alignment error	$\approx 4.0 \times 10^{-11}$
Best after-EvalMod alignment error	$\approx 2.0 \times 10^{-5}$
Direct final max error vs plaintext	$\approx 7 \times 10^{-8}$
Factored RAW max error vs plaintext	$\approx 7 \times 10^{-7}$
Direct vs factored RAW	$\approx 7 \times 10^{-7}$
Direct vs factored DIV-2	$\approx 5.2 \times 10^{-1}$
Observed normalization	RAW is best, $\gamma \approx 1$
Direct output level	10
Factored output level	10
Direct full time	$\approx 117.2s$
Factored full time	$\approx 23.3s$
Speedup	$\approx 5.03\times$

8.2 $\log N = 16, k = 2, \log n = 15$

指标	数值
top $\log N$	16
leaf $\log n$	15
k	2
$\log(QP)$	1520.0
Ring basis compatibility	PASS
Best C2S alignment error	$\approx 1.8 \times 10^{-11}$
Best after-EvalMod alignment error	$\approx 9.4 \times 10^{-6}$
Direct final max error vs plaintext	$\approx 3.2 \times 10^{-8}$
Factored RAW max error vs plaintext	≈ 1.05
Factored DIV-2 max error vs plaintext	$\approx 2.4 \times 10^{-7}$
Direct vs factored DIV-2	$\approx 2.4 \times 10^{-7}$
Observed normalization	DIV-2 is best, $\gamma \approx 2$
Direct output level	10
Factored output level	10
Direct full time	$\approx 26.3s$
Factored full time	$\approx 7.22s$
Speedup	$\approx 3.64\times$

9 参数选择流程：从应用需求到 N, Q, n **9.1 Step 1: 确定应用需求**

先回答：

1. 需要多少 slots?
2. 需要多少 precision?
3. 需要多少 multiplication depth?
4. 是否需要 bootstrapping?
5. 是追求 latency 还是 throughput?

如果是 encrypted neural network inference / training, 通常更偏 throughput, 需要大量 slots 和 bootstrapping。

9.2 Step 2: 确定 scale Δ

常见选择：

$$\Delta = 2^{35} \quad \text{或} \quad 2^{40}.$$

如果希望高精度，选 2^{40} ；如果希望省 modulus，选 2^{35} 或更小。

9.3 Step 3: 估计 Q

如果有 L 层乘法，每层 prime 大约接近 $\log \Delta$ ，则：

$$\log Q \approx \log q_0 + L \log \Delta + \text{bootstrapping primes}.$$

bootstrapping 参数中还要考虑 C2S、EvalMod、S2C、ModUp 等消耗。

9.4 Step 4: 加上 P 得到 QP

如果使用 key switching auxiliary primes：

$$\log P \approx 4 \times 61 = 244 \quad \text{或} \quad 5 \times 61 = 305.$$

最终要估：

$$\log(QP) = \log Q + \log P.$$

9.5 Step 5: 根据 $q_{\max}$ 选择 leaf n

这是你的方案最重要的一步。

如果：

$$\log(QP) \approx 1520,$$

查表得到：

$$2^{15} \text{ 只支持约 } 868,$$

$$2^{16} \text{ 支持约 } 1747.$$

所以 leaf 至少应该选：

$\log n = 16.$

9.6 Step 6: 根据 k 得到 top N

如果 leaf 固定为 $\log n = 16$:

$$k = 2 \Rightarrow \log N = 17,$$

$$k = 4 \Rightarrow \log N = 18.$$

9.7 Step 7: 跑 estimator

最终论文不能只靠表。应报告:

$$\lambda_{\text{scheme}} = \min\{\lambda_{\text{top}}, \lambda_{\text{leaf}}, \lambda_{\text{split}}, \lambda_{\text{merge}}, \lambda_{\text{evalkeys}}\}.$$

并列出:

component	ring degree	$\log q_{\max}$	secret	σ	estimated bits
top ciphertext/key	2^{17}	1520	ternary	3.19	TBD
leaf BTS key	2^{16}	1520	ternary	3.19	TBD
split key	?	?	?	?	TBD
merge key	?	?	?	?	TBD

10 $no-B_k$ 分解中的安全性表达

你的方案可以写作:

$$\text{BTS}_N^{\text{fac}} = \text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} \text{BTS}_n^{(r)} \right) \circ \text{Split}_k.$$

其中:

$$n = N/k.$$

安全性不是只看 top ring:

$$\text{RLWE}_{N,Q}.$$

还必须看 leaf ring:

$$\text{RLWE}_{n,Q_{\text{leaf}}P_{\text{leaf}}}.$$

因此:

$$\boxed{\lambda_{\text{scheme}} = \min_i \lambda_i.}$$

如果任何 leaf 或 switching key 只有 60-bit, 那么整体就是 60-bit, 不是 128-bit。

11 参数和安全性的常见误区

11.1 误区 1: top N 安全, 所以 leaf n 也安全

错误。leaf 是小环实例, 必须单独估。

11.2 误区 2: 只看输出 ciphertext level 的 Q

错误。evaluation keys、rotation keys、bootstrapping keys 可能暴露更大的 QP 模数, 应保守看最大公开模数。

11.3 误区 3: N 越大只是消息更多

不完整。 N 更重要的是提供安全预算来支持更大的 Q 或 QP 。

11.4 误区 4: speedup 应该最多是 k

不一定。direct bootstrapping 在大 N 下可能超线性变慢, 包含 C2S/S2C、rotations、key switching、NTT 和 memory movement。把一个大任务换成多个小 leaf 任务并行, speedup 可以超过 k 。

11.5 误区 5: normalization factor 永远是 k

不是。你的实验已经说明:

$$\gamma_{\text{impl}} = \frac{\text{C2SScaling}_{\text{leaf}} \text{S2CScaling}_{\text{leaf}}}{\text{C2SScaling}_{\text{top}} \text{S2CScaling}_{\text{top}}}.$$

对 $\log N = 17, k = 2, \gamma = 1$; 对 $\log N = 16, k = 2, \gamma = 2$ 。这属于 implementation convention, 不属于安全参数本身。

12 推荐的实验命令

12.1 安全版 $k = 2$

```
go run . \
-logN=17 \
-layers=1 \
-leafOrder=bitrev \
-parallelLeaves=true \
-leafWorkers=2 \
-reps=1 \
-warmup=0
```

这对应:

$$\log N_{\text{top}} = 17, \quad k = 2, \quad \log n_{\text{leaf}} = 16.$$

12.2 安全版 $k = 4$

```
go run . \
-logN=18 \
-layers=2 \
-leafOrder=natural \
-parallelLeaves=true \
-leafWorkers=4 \
-reps=1 \
-warmup=0
```

这对应：

$$\log N_{\text{top}} = 18, \quad k = 4, \quad \log n_{\text{leaf}} = 16.$$

这组会很重。

13 Go 代码：打印参数和安全相关摘要

建议在程序中打印：

```
func printSecurityRelevantParams(name string, logN int, logQP float64, logQ float64,
    logP float64) {
    N := 1 << logN
    slots := N / 2
    fmt.Printf("\n===%s_security-relevant_parameters===\n", name)
    fmt.Printf("LogN=====_%d\n", logN)
    fmt.Printf("N=====_%d\n", N)
    fmt.Printf("Slots=====_%d\n", slots)
    fmt.Printf("logQ=====_%d\n", logQ)
    fmt.Printf("logP=====_%d\n", logP)
    fmt.Printf("logQP=====_%d\n", logQP)
}
```

对你的实验，至少打印：

```
printSecurityRelevantParams("TOP", topLogN, paramsTop.LogQP(), logQTop, logPTop)
printSecurityRelevantParams("LEAF", leafLogN, paramsLeaf.LogQP(), logQLeaf, logPLeaf)
```

并在 summary 中给出：

$$\log N_{\text{top}}, \quad \log n_{\text{leaf}}, \quad k, \quad \log Q, \quad \log P, \quad \log(QP).$$

14 最终建议

当前阶段建议你吧实验分成两类：

14.1 Correctness / mechanism prototype

这些参数可以小一些:

$$\log N = 16, \quad k = 4, \quad \log n = 14.$$

用途:

- 验证 no- B_k 交换图;
- 验证 C2S/EvalMod/S2C 拆分正确性;
- 验证 normalization trace;
- 做快速 timing。

但不声称 128-bit security。

14.2 Security-aware prototype

推荐:

$$\log N = 17, \quad k = 2, \quad \log n = 16, \quad \log(QP) \approx 1520.$$

用途:

- 展示更接近 128-bit 参数的 no- B_k 分解;
- leaf ring 维度足够承受 bootstrapping 级别模数;
- 和 direct $\log N = 17$ 做公平算法对比;
- 报告 output level、precision、speedup。

14.3 Full secure high-throughput target

推荐:

$$\log N = 18, \quad k = 4, \quad \log n = 16.$$

用途:

- 展示高吞吐 / 大 batch 场景;
- leaf 仍为安全常见维度;
- $k = 4$ 并行潜力更大。

缺点是计算和内存会很重。

15 一句话总结

N 是安全预算和容量预算, QP 是消耗安全预算, $n = N/k$ 后必须重新估 leaf 安全。

对当前 no- B_k bootstrapping:

$\log(QP) \approx 1520$ 时, leaf 最好至少取 $\log n = 16$.

因此:

$k = 2 \Rightarrow \log N_{\text{top}} = 17, \quad k = 4 \Rightarrow \log N_{\text{top}} = 18.$

参考资料

1. HomomorphicEncryption.org, *Homomorphic Encryption Standard*. <https://homomorphicencryption.org/standard/>
2. Albrecht et al., *Security Guidelines for Implementing Homomorphic Encryption*, ePrint 2024/463. Table 5.2 gives maximal $\log_2(q)$ for dimensions and security categories. <https://eprint.iacr.org/2024/463>
3. Lattigo v6 bootstrapping package documentation. It states that bootstrapping parameters have independent cryptographic parameters and users must ensure target security. <https://pkg.go.dev/github.com/tuneinsight/lattigo/v6/circuits/ckks/bootstrapping>
4. 当前实验日志: dftparams_logN17_k2.txt, dftparams_logN16_k2.txt, normtrace_logN17_k2.txt, normtrace_logN16_k2_timing.txt.